

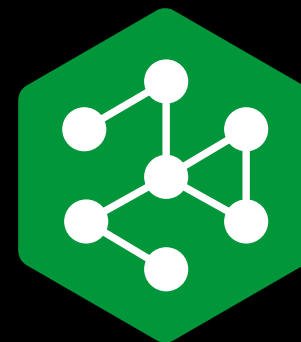


Kubernetes Gateway API and NGINX Gateway Fabric

Dave McAllister

Senior Open Source Technologist

Oct 2025



NGINX
Open Source
Gateway Fabric

Texas Linux Festival October 4th, 2025

Agenda

The Ingress Era

The Gateway API Era

Ingress versus Gateway API

NGINX Gateway Fabric

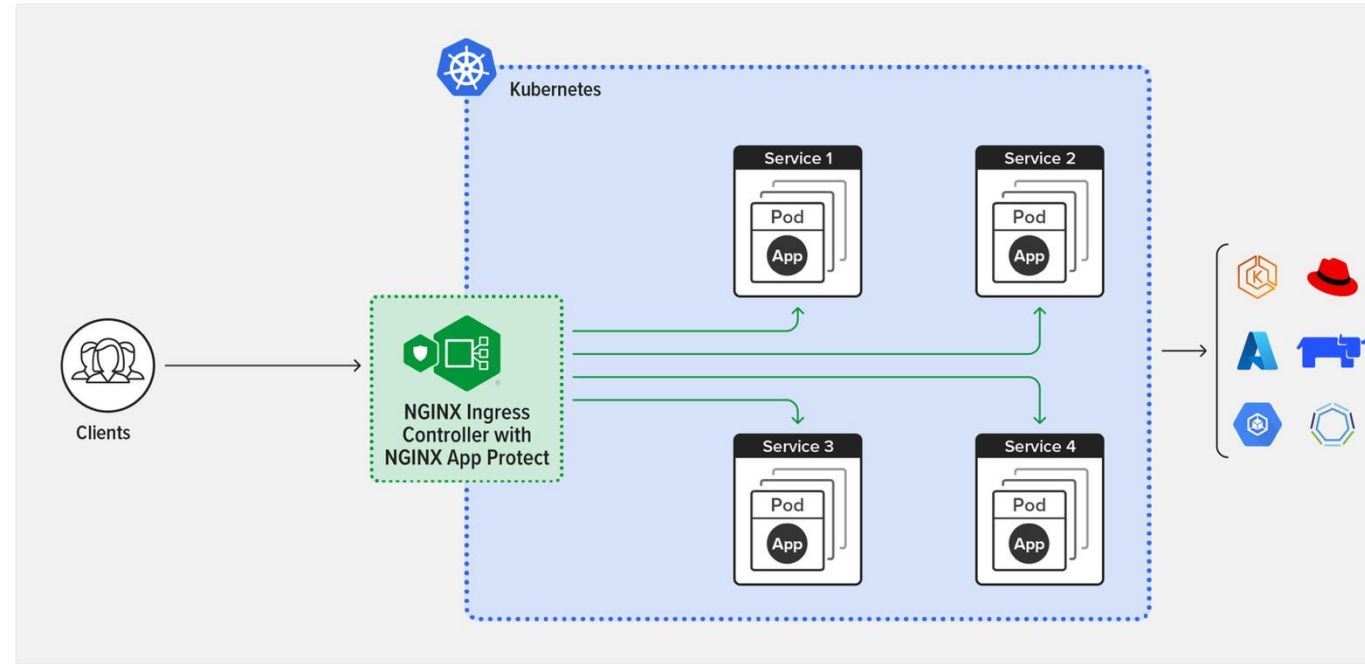
Migration

Real World Considerations



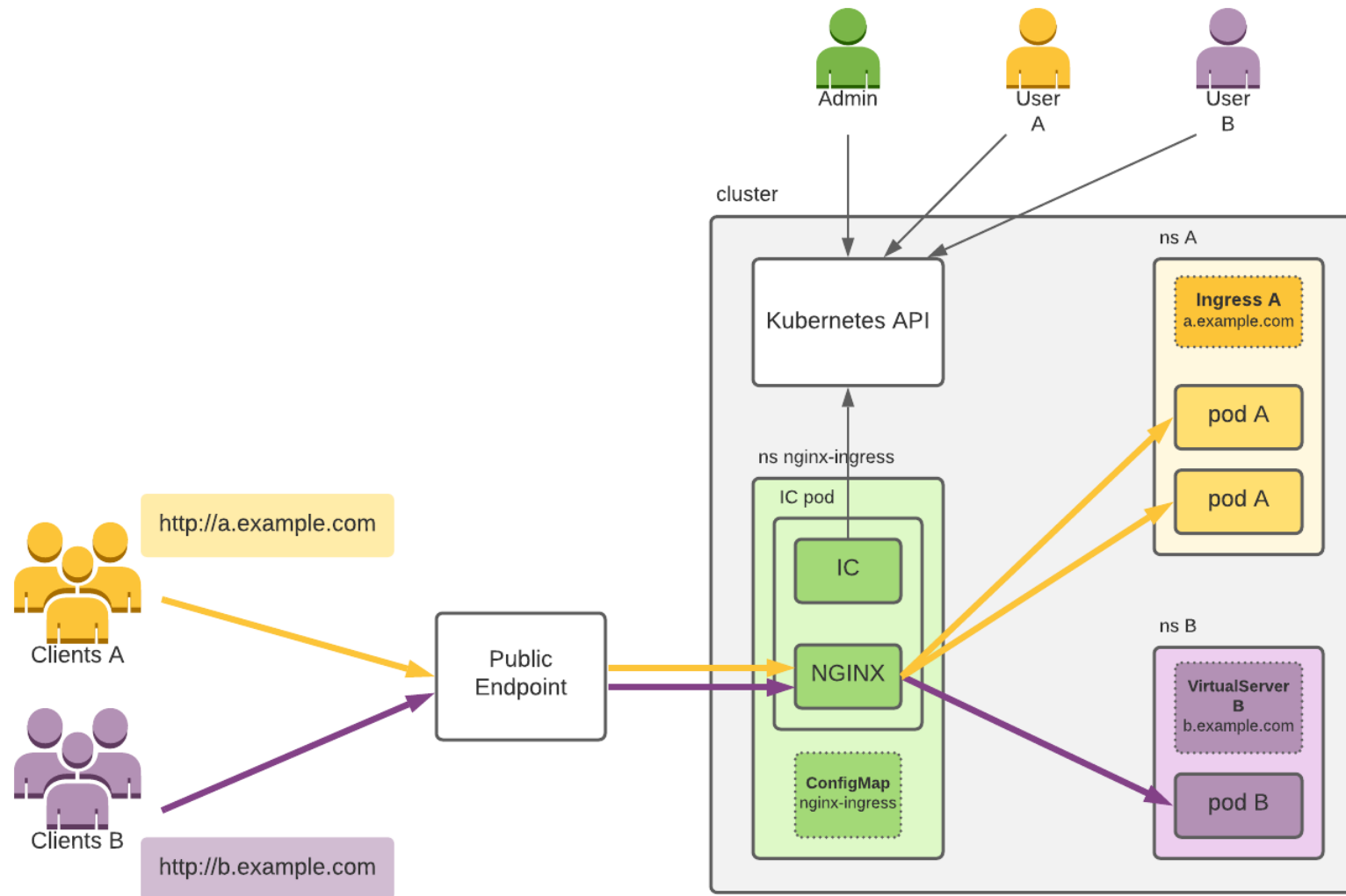
The Ingress Era

- What is Ingress
 - API resource for managing external access
 - HTTP/HTTPS routes
- Two main parts
 - Ingress resource
 - Ingress Controller



Pros	Cons
Easy and simple	Single API = single persona
Well-established “standard”	Limited functionality
	Not (easily) portable

A Simple Example



There are a lot of different Ingress Controllers



NGINX
Open Source
Ingress Controller

NGINX Ingress Controller

open source at <https://github.com/nginx/Kubernetes-ingress>



Community ingress-nginx

The Gateway API Era

- Collection of resources for modeling service networking in Kubernetes
 - Load balancing
 - Traffic routing
 - Ingress

Why?

To provide a collection of resources for modeling service networking in Kubernetes, including load balancing, traffic routing, and ingress for modern environments

- Separate the concerns of different users
- Support advanced routing
- Be portable
- Allow straightforward extension (points)
- Add security features



What is the Kubernetes Gateway API?

- **Open source project** (<https://gateway-api.sigs.k8s.io/>)
- **Kubernetes API add-on** providing infrastructure provisioning and advanced traffic routing
- **Role-oriented** – modeled after organizational roles
 - Infrastructure provider
 - Cluster operator
 - Application developer
- **Portable** – based on custom resources (CRD) and supported by multiple implementations
- **Expressive** – use cases include traffic matching, weighting and routing
- **Extensible** – support granular customization

Some Key Concepts

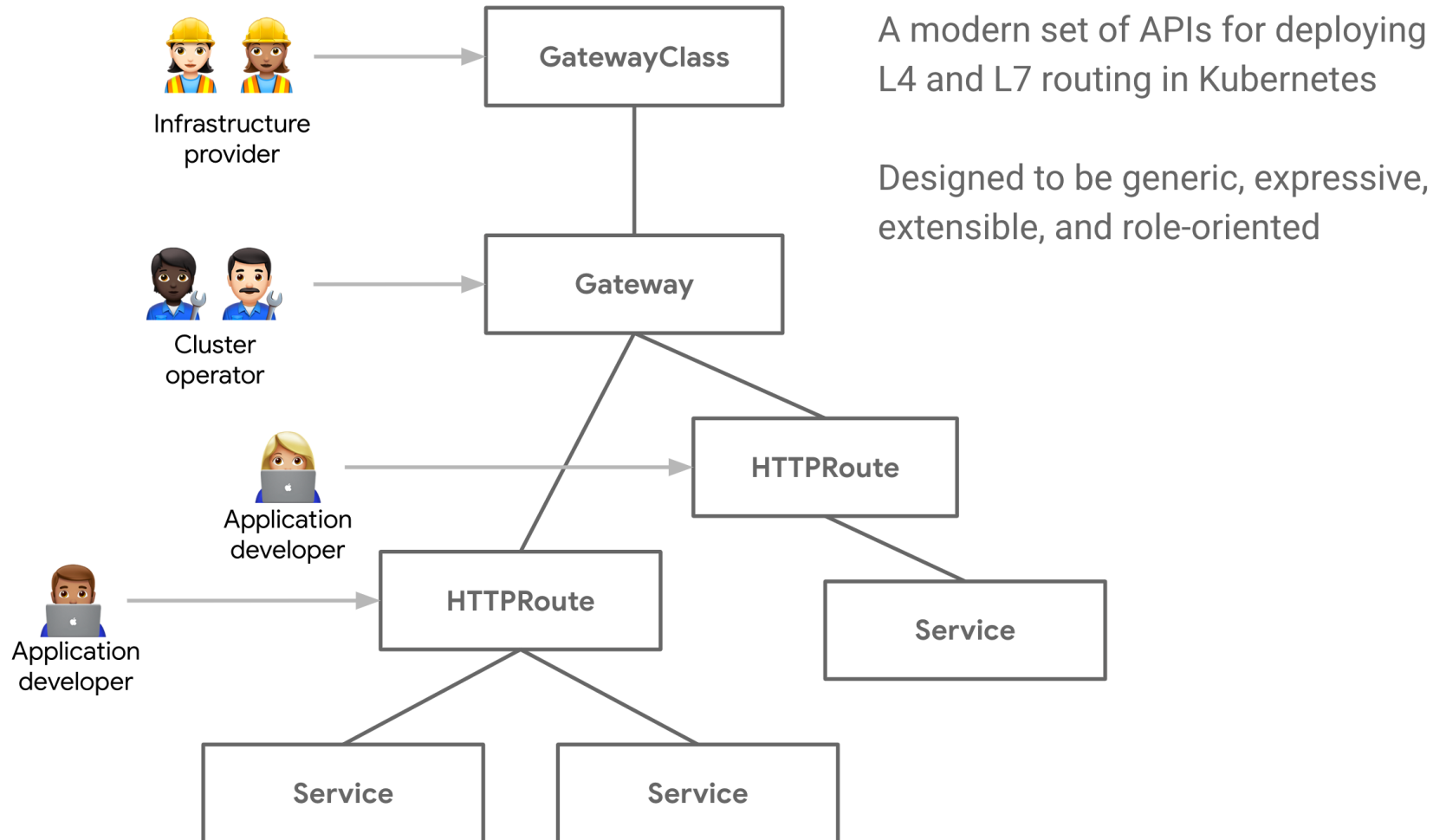
Role-Based Model

- **GatewayClass:** (Infrastructure Provider) Defines a class of gateways. Cluster operators manage this
- **Gateway:** (Cluster Operator) A template for a load balancer. It defines ports and protocols
- **HTTPRoute/TCPRoute/etc.:** (Application Developer) Defines specific routing rules and attaches to a Gateway

Key Features

- **Role-based:** Separates responsibilities between infrastructure providers, cluster operators, and application developers
- **Portability:** Built-in, standardized support for advanced features (traffic splitting, header routing) means less reliance on annotations
- **Extensibility:** Policy attachment and extension points allow for vendor-specific features without breaking portability
- **Multi-tenancy:** The role separation makes it easier and safer to run multiple teams on a single cluster

Kubernetes Gateway API



Two Basic parts

- API Specification
Set of Custom Resource Definitions that define the API Objects
- Controller (or Implementation)
The app that runs in the cluster to watch for changes and implements them on the dataplane
- NGINX offers an open source controller
 - NGINX Gateway Fabric (<https://github.com/nginx/nginx-gateway-fabric>)

Pros	Cons
Role-oriented Design (separates concerns)	Increased complexity
Standardized and Portable	Early in adoption cycle
Feature-rich and extensible	Configuration overhead

Comparing Solutions

Feature	Ingress	Gateway API
Role Separation	Single resource, Single persona	Multiple resources, Clear roles
Portability	Low (vendor specific annotations)	High (standardized features)
Advanced Features	Requires annotations	Natively supported (e.g. traffic splitting)
Protocol Support	HTTP, HTTPS	HTTP, HTTPS, TCP, UDP, TLS
Traffic Flow	Simple path/host routing	Declarative routing
Extensibility	Annotations	Policy attachment, Extension points

NGINX Gateway Fabric (yes, open source)

- High Performance Kubernetes native implementation of the Gateway API
- Focuses on NGINX values
 - Performance
 - Reliability
 - Scalability
 - Security
- Fully compliant with the Gateway API spec
- Adds custom extensions to expose advanced NGINX features
- Designed to be easy to deploy and manage



NGINX
Open Source
Gateway Fabric

Consists of Two Parts

- Control Plane: Centralized Brain
 - Kubernetes Controller (Deployment) manages all configuration.
 - Dynamic Provisioning: Automatically creates a dedicated NGINX Data Plane (Deployment/Service) for every new Gateway resource.
 - Secure Configuration: Translates K8s resources into NGINX config and delivers it securely via gRPC.
- Data Plane: Autonomous Traffic Engine
 - Dedicated NGINX/Agent pods (Deployment or DaemonSet).
 - Receives config via gRPC and handles NGINX reloads internally.
 - 1:1 Mapping: Each Gateway resource maps to one dedicated, isolated data plane.
 - Supports multiple, concurrent gateways for isolation and flexible operations.

Migrating from Ingress to Gateway API

Moving from an Ingress controller to a Gateway API like NGINX Gateway fabric can be challenging

What is ingress2gateway?

- A tool that automatically translates existing Kubernetes Ingress resources to the new Gateway API resources
- A community-driven project within the Gateway API SIG

Why use it?

- **Simplify migration:** Automates the conversion process, saving time and reducing manual errors
- **Preserve configurations:** Aims to map Ingress rules to Gateway API rules

How it works:

- Reads Ingress resources from your cluster or a YAML file
- Generates equivalent Gateway and HTTPRoute YAML
- Supports multiple Ingress providers

NGINX Provider for ingress2gateway

Annotation-specific conversion for NGINX Ingress Controller to NGINX Gateway Fabric

Resource Conversion: ingress2gateway can convert core Kubernetes Ingress resources and associated Services into their Gateway API equivalents.

Annotation Support: The tool supports various NGINX-specific annotations, translating them into appropriate Gateway API resources

Some of the supported annotations include:

nginx.org/ssl-services for SSL/TLS backend connections, mapped to BackendTLSPolicy

nginx.org/grpc-services for gRPC backend connections, mapped to GRPCRoute

nginx.org/websocket-services for WebSocket backend connections, providing an informational notification

nginx.org/proxy-hide-headers and nginx.org/proxy-set-headers for modifying headers, mapped to HTTPRoute ResponseHeaderModifier and RequestHeaderModifier respectively

nginx.org/rewrites for URL rewriting, mapped to HTTPRoute URLRewrite filter

nginx.org/redirect-to-https and ingress.kubernetes.io/ssl-redirect for SSL/HTTPS redirects, mapped to HTTPRoute RequestRedirect filter.

[Full list of annotations to Gateway API mapping](#)

Benefits of Using ingress2gateway

Simplified Migration:

Automates the conversion process, reducing the complexity and effort

Consistency and Accuracy:

Conversion is consistent and accurate

Support for Advanced:

Translates advanced NGINX Ingress Controller features into their Gateway API counterparts

Quick example

```
./ingress2gateway print --providers=nginx --input-file=ingress.yaml > gateway-api.yaml
```

```
# Ingress resource (ingress.yaml)
1 apiVersion: networking.k8s.io/v1
2 kind: Ingress
3 metadata:
4 name: example-ingress
5 annotations:
6 nginx.ingress.kubernetes.io/rewrite-target: /
7 spec:
8 ingressClassName: nginx
9 rules:
10 - host: example.com
11 http:
12 paths:
13 - path: /
14 pathType: Prefix
15 backend:
16 service:
17 name: example-service
18 port:
19 number: 80
```

```
# Converted Gateway API resources (gateway-api.yaml)
1 apiVersion: gateway.networking.k8s.io/v1
2 kind: Gateway
3 metadata:
4 annotations:
5 gateway.networking.k8s.io/generator:
6 ingress2gateway-v0.4.0-rc1-16-g4ceecb
7 name: nginx
8 spec:
9 gatewayClassName: nginx
10 listeners:
11 - hostname: example.com
12 name: example-com-http
13 port: 80
14 protocol: HTTP
15 status: {}
16 ---
17 apiVersion: gateway.networking.k8s.io/v1
18 kind: HTTPRoute
19 metadata:
20 annotations:
```

```
20 gateway.networking.k8s.io/generator:
21 ingress2gateway-v0.4.0-rc1-16-g4ceecb
22 name: example-ingress-example-com
23 spec:
24 hostnames:
25 - example.com
26 parentRefs:
27 - name: nginx
28 rules:
29 - backendRefs:
30 - name: example-service
31 port: 80
32 matches:
33 - path:
34 type: PathPrefix
35 value: /
36 status:
37 parents: []
```

Migrating from Ingress Controller to Gateway API

Migrating to the Gateway API requires careful planning

Pre-Migration Planning:

Document the current setup, including Ingress objects and annotations. Define migration goals and success criteria.

Installation and Configuration:

Deploy NGINX Gateway Fabric alongside the existing Ingress controller. Convert Ingress configurations to Gateway API resources.

Testing and Validation:

Conduct thorough functional and performance testing. Validate security features and monitor performance metrics.

Traffic Cutover:

Gradually shift traffic to NGINX Gateway Fabric using strategies like canary releases.

Post-Migration Cleanup:

Decommission the old Ingress controller after confirming stability.

[Migrating from F5 NGINX Ingress Controller to the F5 NGINX Gateway Fabric – NGINX Community Blog](#)

Some Considerations for Gateway API

CRD Installation:

Gateway API resources are custom resources (CRDs) that must be installed in the cluster

Controller Dependency:

You need a controller (like NGINX Gateway Fabric) running in your cluster to make the Gateway API work

Migration Strategy:

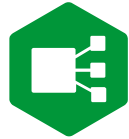
Plan your migration. Do you want to run Ingress and Gateway API side-by-side? How will you handle DNS cutover?

Ecosystem Maturity:

While the core API is GA, some advanced features or integrations may still be in development depending on the provider

NGINX in Kubernetes

NGINX Ingress Controller



- Implements the Ingress API
- Mature product (first release November 2016)
- Near ubiquitous use within Kubernetes clusters
- Extensive capabilities with custom resources and annotations

NGINX Gateway Fabric



- Implements the Gateway API
- Evolution of Ingress
- Stand-alone product built for the Gateway API
- Architected to unify connectivity to, from, and within a Kubernetes cluster

• Summary:

- Ingress: Simple, effective for basic use cases, but limited and not portable
- Gateway API: The future of Kubernetes traffic management. Role-based, extensible, and portable
- NGINX Gateway Fabric: A high-performance implementation of Gateway API
- ingress2gateway: The assist for a smooth migration

[Your Choice: From Ingress Controller to the Gateway API – NGINX Community Blog](#)



**Want to troubleshoot configs
and connect with other
open source enthusiasts?**



**Join the NGINX
Community Forum**

Thanks!
<https://linkedin.com/in/davemc>

Slides on Github:
`dwmcallister/presentation-decks`

